

一次性模仿学习

Yan Duan^{†§}, Marcin Andrychowicz[‡], Bradly Stadie^{†‡}, Jonathan Ho^{†§},
Jonas Schneider[‡], Ilya Sutskever[†], Pieter Abbeel^{†§}, Wojciech Zaremba[‡]

[†]Berkeley AI Research Lab, [‡]OpenAI

[§]Work done while at OpenAI

{rockyduan, jonathanho, pabbeel}@eecs.berkeley.edu

{marcin,bstadie, jonas, ilyasu, woj}@openai.com

摘要

模仿学习通常被应用于孤立地解决不同任务。这通常需要仔细的特征工程或大量的样本。这远非我们所期望的：理想情况下，机器人应该能够从任何给定任务的极少量演示中学习，并立即泛化到同一任务的新情况，而无需特定任务的工程。在本文中，我们提出了一种元学习框架来实现这种能力，我们称之为一次性模仿学习。具体来说，我们考虑存在一个非常大（可能是无限）的任务集合，每个任务有许多实例化。例如，一个任务可能是将桌子上的所有积木堆成一个塔，另一个任务可能是将桌子上的所有积木放置成两个积木的塔，等等。在每种情况下，任务的不同实例将由具有不同初始状态的不同积木集合组成。在训练时，我们的算法接收所有任务子集的演示对。训练一个神经网络，使其在输入第一个演示和从第二个演示中采样的状态时，能够预测与采样状态对应的动作。在测试时，呈现新任务单个实例的完整演示，并期望神经网络在该新任务的新实例上表现良好。我们的实验表明，使用软注意力使模型能够泛化到训练数据中未见过的条件和任务。我们预计，通过在更多种类的任务和设置上训练该模型，我们将获得一个通用系统，可以将任何演示转化为能够完成极其多样任务的稳健策略。

1 引言

我们关注的是能够执行各种复杂有用任务的机器人系统，例如整理家居或准备餐食。机器人应当能够学习新任务，而无需长时间的系统交互。为实现这一目标，我们必须解决两大问题。第一个问题是灵巧性：机器人应学会如何接近、抓取和拾起复杂物体，以及如何将它们放置或排列成期望的构型。第二个问题是沟通：如何传达当前任务的意图，使机器人能够在更广泛的初始条件下复现该任务。演示是一种极为便捷的信息形式，可用于教导机器人克服这两大挑战。通过演示，我们可以明确地传达几乎任何操作任务，同时提供执行任务所需的具体运动技能的线索。我们可以将其与另一种沟通形式——自然语言——进行比较。尽管语言高度通用、有效且高效，但自然语言处理

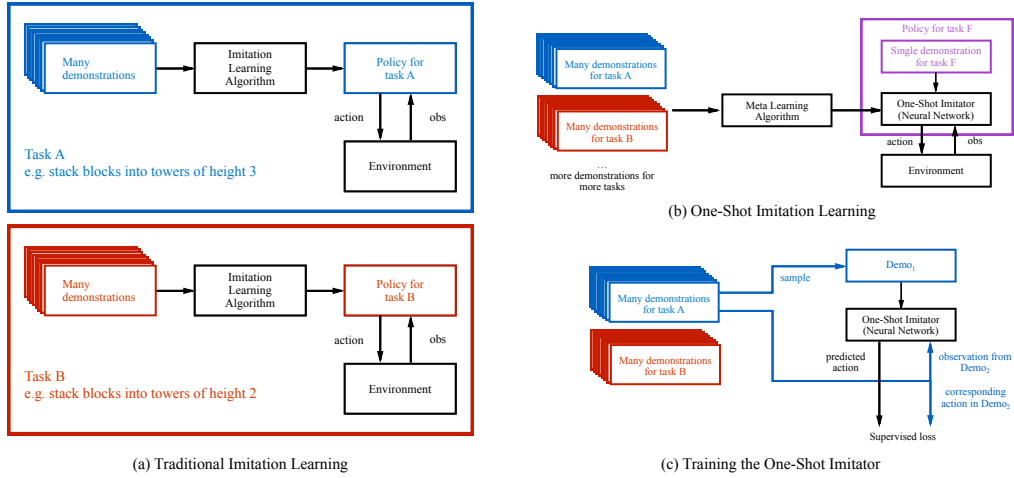


图 1: (a) 传统上, 策略是任务特定的。例如, 一个策略可能通过模仿学习算法训练, 将积木堆成高度为 3 的塔, 然后另一个策略将被训练将积木堆成高度为 2 的塔, 等等。(b) 在本文中, 我们关注训练不特定于单一任务的网络, 而是可以通过单个演示告知当前新任务是什么, 并成功完成该新任务。例如, 当以任务 F 的单个演示为条件时, 它应表现得像任务 F 的良好策略。(c) 我们可以将其表述为监督学习问题, 在一组训练任务上训练该网络, 并且通过足够的示例, 它应泛化到未见但相关的任务。为训练该网络, 在每次迭代中, 我们从其中一个训练任务中采样一个演示, 并将其输入网络。然后, 我们从同一任务的第二个演示中采样另一对观测和动作。当以第一个演示和该观测为条件时, 网络被训练输出相应的动作。

系统尚未达到能够轻松使用语言向机器人精确描述复杂任务的水平。与语言相比, 使用演示具有两个根本优势: 首先, 它不需要语言知识, 因为可以向不说同一种语言的人类传达复杂任务。其次, 即使假设具备完美的语言能力, 也有许多任务极难用言语解释: 例如, 在没有演示和经验的情况下解释如何游泳, 似乎至少是一项极具挑战性的任务。事实上, 从演示中学习已经有许多成功的应用。然而, 迄今为止, 这些应用要么需要仔细的特征工程, 要么需要大量的系统交互时间。这远非我们所期望的: 理想情况下, 我们希望仅向机器人演示一次或几次特定任务, 就能使其立即泛化到同一任务的新情境中, 而无需长时间的系统交互或关于单个任务的领域知识。本文探索了图 1 所示的一次性模仿学习设置, 其目标是在面对新的、未见过的任务时, 仅接收该任务的一次演示作为输入, 最大化所学策略的预期性能。对于我们所考虑的任务, 策略在接收到演示后, 预期无需任何额外的系统交互即可实现良好性能。我们在广泛的任务分布上训练策略, 其中任务数量可能是无限的。对于每个训练任务, 我们假设有一组成功的演示可用。我们学到的策略以以下内容作为输入:

(i) 当前观测, 以及 (ii) 成功解决同一任务不同实例的一次演示 (该演示在整个回合期间保持不变)。策略输出当前控制量。我们注意到, 同一任务的任意一对演示为神经网络策略提供了一个监督训练样本, 其中一个演示被视为输入, 另一个被视为输出。

为使该模型有效运行，本文关键性地采用了软注意力机制（Soft Attention）[6]，既用于处理与演示相对应的（可能较长的）状态与动作序列，也用于处理指定环境中各积木位置向量的各个分量。对这两类输入均使用软注意力机制，使得强大的泛化能力成为可能。特别地，在一系列积木堆叠任务中，本文的神经网络策略能够在训练数据中未曾出现的新颖积木配置上表现良好。实验视频可在 <http://bit.ly/nips2017-oneshot> 获取。

2 相关工作

模仿学习考虑从观察示范中获取技能的问题。综述文章包括 [48, 11, 3]。模仿学习中的两条主要工作线是行为克隆，它从观察到动作执行监督学习（例如，[41, 44]）；以及逆强化学习 [37]，其中估计一个奖励函数 [1, 66, 29, 18, 22]，将示范解释为（接近）最优行为。虽然过去的工作已经产生了广泛的令人印象深刻的机器人成果，但它分别考虑每个技能，并且学会模仿一个技能并不会加速学习模仿下一个技能。一次性学习和少样本学习已被研究用于图像识别 [61, 26, 47, 42]、生成建模 [17, 43] 以及使用循环策略学习“快速”强化学习智能体 [16, 62]。快速适应也已通过快速权重实现 [5]。与我们的算法一样，上述许多方法都是元学习的一种形式 [58, 49, 36]，其中算法本身正在被学习。元学习也已被研究用于发现神经网络权重优化算法 [8, 9, 23, 50, 2, 31]。然而，先前关于一次性学习和元学习的工作是针对各自领域（图像识别、生成模型、强化学习、优化）量身定制的，并不直接适用于模仿学习环境。最近，[19] 提出了一个通用框架，用于跨上述几个领域进行元学习。然而，他们没有考虑模仿学习环境。强化学习 [56, 10] 通过试错学习提供了技能获取的替代途径。强化学习已经取得了许多成功，包括西洋双陆棋 [57]、直升机控制 [39]、雅达利 [35]、围棋 [52]、仿真中的连续控制 [51, 21, 32] 以及真实机器人上的连续控制 [40, 30]。然而，强化学习往往需要大量的试验，并且需要指定奖励函数来定义手头的任务。前者可能耗时，而后者通常比提供示范要困难得多 [37]。多任务和迁移学习考虑学习具有超出单一任务的适用性和重用性的策略的问题。成功案例包括计算机视觉 [64, 34, 28, 4, 15, 24, 33, 59, 14] 和控制 [60, 45, 46, 20, 54] 中的领域自适应。然而，虽然比独立获取每个技能更快地获取多种技能，但这些方法并不能提供从单个示范中轻松掌握新技能的能力。我们的方法严重依赖于对示范的注意力模型和对当前观察的注意力模型。我们使用 [6] 中提出的用于机器翻译的软注意力模型，该模型在图像描述生成 [63] 中也取得了成功。[7, 12] 中提出的交互网络也在学习中利用了物理交互的局部性。我们的模型也与序列到序列模型 [55, 13] 相关，因为在两种情况下，我们都消耗一个非常长的示范序列，并有效地发出一个长的动作序列。

3 单样本模仿学习

3.1 问题形式化

本文以 $\mathbb{T}$ 表示任务分布，以 $t \sim \mathbb{T}$ 表示单个任务，以 $\mathbb{D}(t)$ 表示任务 t 的演示分布。策略由 $\pi_{\theta}(a|o, d)$ 表示，其中 a 为动作， o 为观测， d 为演示， θ 为策略参数。演示 $d \sim \mathbb{D}(t)$ 是观测与动作的序列： $d = [(o_1, a_1), (o_2, a_2), \dots, (o_T, a_T)]$ 。本文假设任务分布 $\mathbb{T}$ 已知，且能够为每个任务获取成功演示。本文假设每个任务存在某个标量值评估函数 $R_t(d)$ （例如二值

指示成功与否)，尽管训练期间并不需要该函数。目标在于最大化策略的期望性能，其中期望针

$$d \in \mathbb{D}(t), \text{ 对任务 } t \in \mathbb{T}, \text{ 与演示求取}$$

3.2 积木堆叠任务

为阐明问题设定，本文描述一个积木堆叠任务分布的具体示例，后续实验也将对此进行研究。这些任务共享的组合结构使我们能够研究对未见任务的非平凡泛化。每个任务的目标是控制一个 7 自由度 Fetch 机械臂，将不同数量的立方体积木堆叠成用户指定的特定构型。每个构型由排列成不同高度塔的积木列表组成，并可用字符串标识。例如，ab cd ef gh 表示我们希望堆叠 4 座塔，每座塔有两块积木，且希望积木 A 位于积木 B 之上，积木 C 位于积木 D 之上，积木 E 位于积木 F 之上，积木 G 位于积木 H 之上。每种构型对应一个不同的任务。此外，在每个回合中，积木的起始位置可能变化，这要求所学策略即使在训练任务内部也能泛化。在典型任务中，观测是 (x, y, z) 个物体相对于夹爪的位置列表，以及夹爪开合状态信息。不同任务实例中物体数量可能不同。本文将阶段定义为将一块积木堆叠到另一块之上的单次操作。例如，任务 ab cd ef gh 有 4 个阶段。

3.3 算法

为了训练神经网络策略，我们利用行为克隆和 DAGGER 等模仿学习算法 [44]，这些算法只需要示范而不需要指定奖励函数。这有可能更具可扩展性，因为通常示范一个任务比指定一个形状良好的奖励函数更容易 [38]。我们首先为每个任务收集一组示范，在动作中加入噪声以在轨迹空间中获得更广泛的覆盖。在每次训练迭代中，我们采样一个任务列表（有放回）。对于每个采样的任务，我们采样一个示范以及一小批观测-动作对。策略被训练为在当前观测和示范的条件下回归到期望的动作，通过最小化 ℓ_2 或交叉熵损失，具体取决于动作是连续的还是离散的。训练过程的高层图示见图 1(c)。在所有实验中，我们使用 Adamax[25] 进行优化，学习率为

$$0.001.$$

4 架构

虽然原则上通用神经网络可以学习从示范和当前观测到适当动作的映射，但我们发现使用合适的架构很重要。我们用于学习积木堆叠的架构是本文的主要贡献之一，我们相信它代表了未来考虑更复杂任务时一次性模仿学习架构的可能形态。我们提出的架构由三个模块组成：示范网络、上下文网络和操作网络。架构图示见图 2。我们将在下面描述每个模块执行的主要操作，完整规范见附录。

4.1 示范网络

示范网络接收示范轨迹作为输入，并生成示范的嵌入供策略使用。该嵌入的大小随示范长度以及环境中积木数量线性增长。

时间随机失活：对于积木堆叠任务，演示可以跨越数百到数千个时间步，使用如此长的序列进行训练在时间和内存消耗上都可能要求很高。因此，我们在训练期间随机丢弃一部分时间步，这一操作我们称之为时间随机失活（temporal dropout），类似于 [53, 27]。我们用 p 表示被丢弃的时间步的比例。

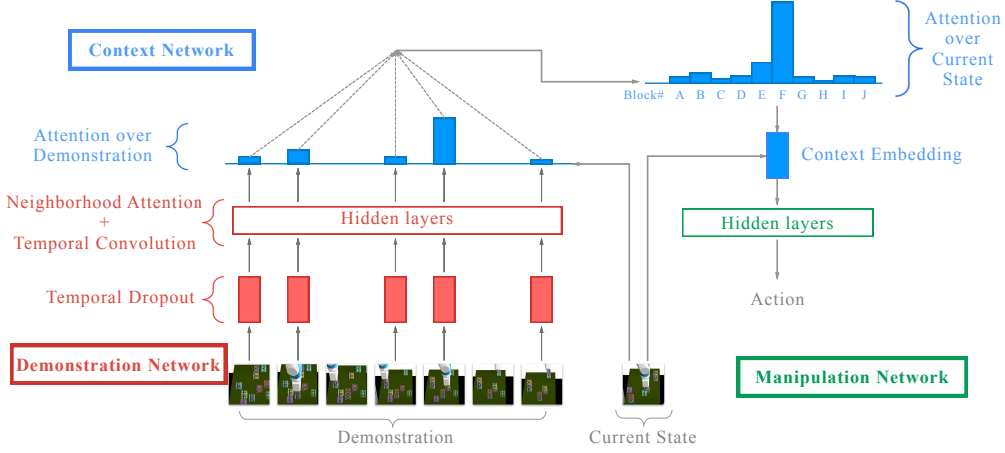


图 2: 网络架构示意图。

在我们的实验中，我们使用 $p = 0.95$ ，这将演示的长度缩短了 20 倍。在测试时，我们可以采样多个下采样轨迹，使用每个轨迹计算下游结果，并对这些结果取平均以产生集成估计。根据我们的经验，这持续地提升了策略的性能。

邻域注意力：在对演示进行下采样之后，我们应用一系列操作，包括膨胀时间卷积 [65] 和邻域注意力。我们现在更详细地描述后一种操作。由于我们的神经网络需要处理具有可变数量积木的演示，它必须拥有能够处理可变维度输入的模块。软注意力是一种自然的操作，它将可变维度输入映射到固定维度输出。然而，这样做可能会丢失相对于其输入的信息。这是不希望，因为演示中包含的信息量随着积木数量的增加而增长。因此，我们需要一种能够将可变维度输入映射到具有可比维度输出的操作。直观地说，与其通过关注所有输入得到单个输出，不如拥有与输入一样多的输出，并让每个输出根据其自身对应的输入来关注所有其他输入。

我们首先描述 [6] 中指定的软注意力模块。注意力的输入包括一个查询 q 、一个上下文向量列表 $\{c_j\}$ 和一个记忆向量列表 $\{m_j\}$ 。第 i 个注意力权重由 $w_i v^T$ 双曲正切函数 $(q + c_i)$ 给出，其中 v 是一个学习到的权重向量。注意力的输出是记忆内容的加权组合，其中权重由注意力权重上的 Softmax 函数操作给出。形式上，我们有输出 $P_i m_i \exp(w_i) / \sum_j \exp(w_j)$ 。注意，输出与记忆向量具有相同的维度。注意力操作可以推广到多个查询头，在这种情况下，输出向量的数量将与查询的数量一样多。

现在转向邻域注意力机制。假设环境中存在 B 个方块。将机器人状态记为 s_{robot} ，每个方块的坐标记为 $(x_1, y_1, z_1), \dots, (x_B, y_B, z_B)$ 。邻域注意力的输入是一组同维度的嵌入列表 $h_1^{in}, \dots, h_B^{in}$ ，该列表可以是对方块位置列表进行投影操作的结果，也可以是前一次邻域注意力操作的输出。给定该嵌入列表，使用两个独立的线性层为每个方块计算查询向量和上下文嵌入： q_i 线性变换 (h_i^{in})，以及 c_i 线性变换 (h_i^{in})。待提取的记忆内容由每个方块的坐标与输入嵌入拼接而成。第 i 个查询结果由以下软注意力操作给出：结果 i 软注意力 (查询: q_i , 上下文: $\{c_j\}_{j=1}^B$, 记忆: $\{((x_j, y_j, z_j), h_j^{in})\}_{j=1}^B$)。直观上，该操作允许每个方块相对于自身查询其他方块 (例如找到最近的方块)，并提取所查询的信息。随后将收集到的结果与每个方块自身的信息相结合，为每个方块生成输出嵌入。具体而言，有

输出 i 线性 (Linear) 拼接 (concat) h_i^{in} 。在实践中, 我们使用多个查询 每个块的头数, 因此每个结果 i 的大小将与查询头数 $\text{result}_i, (x_i, y_i, z_i), s_{\text{robot}}))$ 成正比。

4.2 上下文网络

上下文网络是本模型的核心。它同时处理当前状态和演示网络生成的嵌入, 并输出一个上下文嵌入, 其维度不依赖于演示的长度或环境中块的数量。因此, 它被迫仅捕获相关信息, 这些信息将被操作网络使用。

对演示的注意力: 上下文网络首先根据当前状态计算查询向量, 随后利用该向量对演示嵌入中不同时间步进行注意力操作。同一时间步内不同块的注意力权重被求和, 从而为每个时间步生成单一权重。这种时间注意力的结果是一个向量, 其大小与环境中的块数成正比。接着, 我们应用邻域注意力在每块的嵌入之间传播信息。此过程重复多次, 其中状态通过使用非共享权重的长短期记忆单元进行更新。

对当前状态的注意力: 前述操作生成的嵌入, 其大小与演示长度无关, 但仍取决于块的数量。随后, 我们对当前状态应用标准软注意力机制, 以生成固定维度的向量, 其中记忆内容仅由每个块的位置组成, 这些位置与机器人状态共同构成上下文嵌入, 并传递给操作网络。直观上, 尽管环境中物体的数量可能变化, 但在操作过程的每个阶段, 相关物体的数量较少且通常固定。具体到块堆叠环境, 机器人只需关注其试图抓取的块 (源块) 的位置, 以及其试图放置于其上的块 (目标块) 的位置。因此, 经过适当训练的网络能够学习将当前状态与演示中的相应阶段进行匹配, 并推断源块和目标块的身份, 这些身份以对不同块的软注意力权重表示, 随后用于提取相应位置并传递给操作网络。尽管我们在训练中未强制此解释, 但实验分析支持了所学策略内部工作机制的这一解释。

4.3 操作网络

操作网络是最简单的组件。在提取源块和目标块的信息后, 它使用简单的多层感知机网络计算完成当前阶段将一个块堆叠到另一个块上所需的动作。¹ 这种分工为模块化训练提供了可能: 操作网络可以训练完成这一简单过程, 而无需了解演示或环境中存在超过两个块的情况。我们将这一可能性留待未来工作。

5 实验

我们对第 3.2 节描述的块堆叠任务进行了实验。² 这些实验旨在回答以下问题:

- 使用行为克隆训练与 DAGGER 相比如何?
- 以整个演示为条件与以最终状态为条件相比如何, 即使最终状态已包含足够信息来完全指定任务?
- 以整个演示为条件与以轨迹的“快照”为条件相比如何, 其中快照是最具信息量的一小部分帧?

¹ 原则上, 可以用循环神经网络模块替换该模块。但对于本文所考虑的任务, 我们发现并无此必要。

² 更多实验结果见附录, 包括粒子到达任务的简单示例以及积木堆叠的进一步分析。

- 本文框架能否泛化到训练期间从未见过的任务？

为回答这些问题，我们比较以下架构的性能：

- **BC**：采用与之前相同的架构，但使用行为克隆训练策略。

- **数据集聚合**：采用上一节所述架构，并使用数据集聚合训练策略。
- **最终状态**：该架构以最终状态而非整个演示轨迹为条件。对于积木堆叠任务族，最终状态唯一标识任务，无需额外信息。然而，包含任务解决方案中间阶段信息的完整轨迹，可使最优策略的训练更为容易，因为策略可直接依赖演示，而无需将中间步骤记忆到参数中。这与强化学习中奖励塑形对性能的显著影响相关 [38]。两种条件策略的比较将验证该假设是否成立。本文使用数据集聚合训练该策略。

- **快照**：该架构以轨迹的“快照”为条件，快照包含演示轨迹中每个阶段的最后一帧。这假设在测试时能够获得演示的分段信息，与其他条件策略相比，这赋予了它不公平的优势。因此，它可能比以完整轨迹为条件的策略表现更好，并作为参考，帮助我们了解以整个轨迹为条件的策略能否达到与清晰分段演示相当的性能。同样，我们使用 DAGGER 训练该策略。

我们在训练期间见过的任务以及训练期间未见过的任务上评估该策略。注意，泛化在多个层面进行评估：学习到的策略不仅需要泛化到已见任务的新配置和新演示，还需要泛化到新任务。具体而言，我们收集了 140 个训练任务和 43 个测试任务，每个任务具有不同的期望积木布局。每个任务中的积木数量可以在 2 到 10 之间变化。我们为每个任务收集 1000 条轨迹用于训练，并保留一组独立的轨迹和初始配置用于评估。轨迹使用硬编码策略收集。

5.1 性能评估

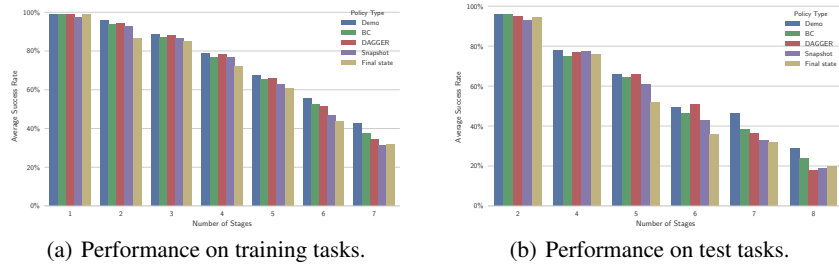


图 3：不同条件策略的比较。最暗的条形表示硬编码策略的性能，不出所料，它在大多数情况下表现最佳。对于使用时间随机失活的架构，我们使用 10 个不同下采样演示的集成，并对动作分布取平均。然后对于所有架构，我们使用贪心动作进行评估。

图 3 展示了各种架构的性能。训练任务和测试任务的结果分别呈现，我们按完成任务所需的阶段数对任务进行分组。这是因为需要更多阶段才能完成的任务通常更具挑战性。事实上，即使我们的脚本策略在最难的任务上也经常失败。我们通过在 100 种不同配置中执行贪心策略（在每个时间步选择置信度最高的动作）来衡量每个任务的成功率，每种配置都以训练期间未见过的不同演示为条件。我们报告同一组内所有任务的平均成功率。

从图中可以观察到，对于阶段较少的简单任务，所有不同的条件策略表现相当且几乎完美。然而，随着难度（阶段数）的增加，以整个演示为条件的策略开始优于以最终状态为条件的策略。一种可能的解释是，当仅以最终状态为条件时，策略可能难以确定应先堆叠哪个积木，而这一信息可以从演示中轻易获取，演示不仅传达了任务，还提供了有助于完成任务的宝贵信息。更令人惊讶的是，以整个演示为条件似乎也优于以快照为条件，而我们原本预期快照表现最佳。我们推测这是由于时间随机失活（temporal dropout）引入的正则化效应，它有效地扩充了策略在训练期间看到的演示集合。另一个有趣的发现是，使用行为克隆（behavioral cloning）训练的性能与使用 DAGGER 训练的性能相当，这表明整个训练过程可以在不需要交互式监督的情况下进行。在我们的初步实验中，我们发现向轨迹收集过程注入噪声对于行为克隆的良好表现至关重要，因此在本文报告的所有实验中我们都使用了噪声注入。在实践中，这种噪声可以来自遥操作中自然的人为噪声，或者在应用于物理机器人之前人为注入额外噪声。

5.2 可视化

我们可视化主策略架构背后的注意力机制，以更好地理解其运作方式。我们主要关注两种注意力：一种是策略关注演示中不同时间步的注意力，另一种是策略关注当前状态中不同积木的注意力。图 4 展示了一些注意力热图。

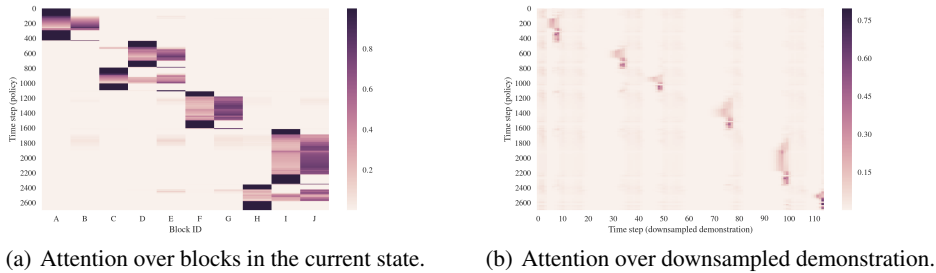


图 4：可视化策略在整个执行过程中执行的注意力。所执行的任务是 ab cde fg hij。请注意，策略对每种注意力类型都有多个查询头，并且每种类型仅可视化一个查询头。（a）我们可以观察到，策略几乎总是关注当前状态中块位置的一小部分，这使得操作网络能够泛化到对不同块的操作。

（b）我们可以观察到具有高注意力权重的时间步的稀疏模式。这表明策略基本上学会了分割演示，并且只关注重要的关键帧。请注意，大约有 6 个高注意力权重区域，这恰好对应于完成任务所需的 6 个阶段。

6 结论

在这项工作中，我们提出了一个简单的模型，该模型将任务的单个成功演示映射到一个有效的策略，该策略在新情况下解决所述任务。我们在一系列块堆叠任务上证明了这种方法的有效性。未来工作有许多令人兴奋的方向。我们计划将该框架扩展到图像数据形式的演示，这将允许更多的端到端学习，而无需单独感知模块。我们也有兴趣使策略能够以多个演示为条件，以防一个演示不能完全解决目标中的歧义。此外，最重要的是，我们希望扩大规模

我们的方法在更大、更广泛的任务分布上，并探索其朝向通用机器人模仿学习系统的潜力，该系统将能够实现各种各样的任务。

7 致谢

我们要感谢加州大学伯克利分校和 OpenAI 的同事们富有洞察力的讨论。这项研究部分由 ONR 通过 PECASE 奖资助。Yan Duan 还得到了华为奖学金的支持。Jonathan Ho 还得到了 NSF 奖学金的支持。

参考文献

- [1] Pieter Abbeel and Andrew Ng. Apprenticeship learning via inverse reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2004.
- [2] Marcin Andrychowicz, Misha Denil, Sergio Gomez, Matthew W Hoffman, David Pfau, Tom Schaul, and Nando de Freitas. Learning to learn by gradient descent by gradient descent. In *Neural Information Processing Systems (NIPS)*, 2016.
- [3] Brenna D Argall, Sonia Chernova, Manuela Veloso, and Brett Browning. A survey of robot learning from demonstration. *Robotics and autonomous systems*, 57(5):469–483, 2009.
- [4] Yusuf Aytar and Andrew Zisserman. Tabula rasa: Model transfer for object category detection. In *2011 International Conference on Computer Vision*, pages 2252–2259. IEEE, 2011.
- [5] Jimmy Ba, Geoffrey E Hinton, Volodymyr Mnih, Joel Z Leibo, and Catalin Ionescu. Using fast weights to attend to the recent past. In *Neural Information Processing Systems (NIPS)*, 2016.
- [6] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. *arXiv preprint arXiv:1409.0473*, 2014.
- [7] Peter Battaglia, Razvan Pascanu, Matthew Lai, Danilo Jimenez Rezende, et al. Interaction networks for learning about objects, relations and physics. In *Advances in Neural Information Processing Systems*, pages 4502–4510, 2016.
- [8] Samy Bengio, Yoshua Bengio, Jocelyn Cloutier, and Jan Gecsei. On the optimization of a synaptic learning rule. In *Optimality in Artificial and Biological Neural Networks*, pages 6–8, 1992.
- [9] Yoshua Bengio, Samy Bengio, and Jocelyn Cloutier. *Learning a synaptic learning rule*. Université de Montréal, Département d’informatique et de recherche opérationnelle, 1990.
- [10] Dimitri P Bertsekas and John N Tsitsiklis. Neuro-dynamic programming: an overview. In *Decision and Control, 1995., Proceedings of the 34th IEEE Conference on*, volume 1, pages 560–564. IEEE, 1995.
- [11] Sylvain Calinon. *Robot programming by demonstration*. EPFL Press, 2009.
- [12] Michael B Chang, Tomer Ullman, Antonio Torralba, and Joshua B Tenenbaum. A compositional object-based approach to learning physical dynamics. In *Int. Conf. on Learning Representations (ICLR)*, 2017.
- [13] Kyunghyun Cho, Bart Van Merriënboer, Caglar Gulcehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.
- [14] Jeff Donahue, Yangqing Jia, Oriol Vinyals, Judy Hoffman, Ning Zhang, Eric Tzeng, and Trevor Darrell. Decaf: A deep convolutional activation feature for generic visual recognition. In *ICML*, pages 647–655, 2014.
- [15] Lixin Duan, Dong Xu, and Ivor Tsang. Learning with augmented features for heterogeneous domain adaptation. *arXiv preprint arXiv:1206.4660*, 2012.

- [16] Yan Duan, John Schulman, Xi Chen, Peter L Bartlett, Ilya Sutskever, and Pieter Abbeel. $R1^2$: Fast reinforcement learning via slow reinforcement learning. *arXiv preprint arXiv:1611.02779*, 2016.
- [17] Harrison Edwards and Amos Storkey. Towards a neural statistician. *International Conference on Learning Representations (ICLR)*, 2017.
- [18] Chelsea Finn, Sergey Levine, and Pieter Abbeel. Guided cost learning: Deep inverse optimal control via policy optimization. In *Proceedings of the 33rd International Conference on Machine Learning*, volume 48, 2016.
- [19] Chelsea Finn, Pieter Abbeel, and Sergey Levine. Model-agnostic meta-learning for fast adaptation of deep networks. *arXiv preprint arXiv:1703.03400*, 2017.
- [20] Abhishek Gupta, Coline Devin, YuXuan Liu, Pieter Abbeel, and Sergey Levine. Learning invariant feature spaces to transfer skills with reinforcement learning. In *Int. Conf. on Learning Representations (ICLR)*, 2017.
- [21] Nicolas Heess, Gregory Wayne, David Silver, Tim Lillicrap, Tom Erez, and Yuval Tassa. Learning continuous control policies by stochastic value gradients. In *Advances in Neural Information Processing Systems*, pages 2944–2952, 2015.
- [22] Jonathan Ho and Stefano Ermon. Generative adversarial imitation learning. In *Advances in Neural Information Processing Systems*, pages 4565–4573, 2016.
- [23] Sepp Hochreiter, A Steven Younger, and Peter R Conwell. Learning to learn using gradient descent. In *International Conference on Artificial Neural Networks*. Springer, 2001.
- [24] Judy Hoffman, Erik Rodner, Jeff Donahue, Trevor Darrell, and Kate Saenko. Efficient learning of domain-invariant image representations. *arXiv preprint arXiv:1301.3224*, 2013.
- [25] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*, 2014.
- [26] Gregory Koch. Siamese neural networks for one-shot image recognition. *ICML Deep Learning Workshop*, 2015.
- [27] David Krueger, Tegan Maharaj, János Kramár, Mohammad Pezeshki, Nicolas Ballas, Nan Rosemary Ke, Anirudh Goyal, Yoshua Bengio, Hugo Larochelle, Aaron Courville, et al. Zoneout: Regularizing rnns by randomly preserving hidden activations. *arXiv preprint arXiv:1606.01305*, 2016.
- [28] Brian Kulis, Kate Saenko, and Trevor Darrell. What you saw is not what you get: Domain adaptation using asymmetric kernel transforms. In *Computer Vision and Pattern Recognition (CVPR), 2011 IEEE Conference on*, pages 1785–1792. IEEE, 2011.
- [29] S. Levine, Z. Popovic, and V. Koltun. Nonlinear inverse reinforcement learning with gaussian processes. In *Advances in Neural Information Processing Systems (NIPS)*, 2011.
- [30] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39):1–40, 2016.
- [31] Ke Li and Jitendra Malik. Learning to optimize. *arXiv preprint arXiv:1606.01885*, 2016.
- [32] Timothy P Lillicrap, Jonathan J Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971*, 2015.
- [33] Mingsheng Long and Jianmin Wang. Learning transferable features with deep adaptation networks. *CoRR, abs/1502.02791*, 1:2, 2015.
- [34] Yishay Mansour, Mehryar Mohri, and Afshin Rostamizadeh. Domain adaptation: Learning bounds and algorithms. *arXiv preprint arXiv:0902.3430*, 2009.

- [35] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [36] Devang K Naik and RJ Mammone. Meta-neural networks that learn by learning. In *International Joint Conference on Neural Networks (IJCNN)*, 1992.
- [37] Andrew Ng and Stuart Russell. Algorithms for inverse reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2000.
- [38] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *ICML*, volume 99, pages 278–287, 1999.
- [39] Andrew Y Ng, H Jin Kim, Michael I Jordan, Shankar Sastry, and Shiv Ballianda. Autonomous helicopter flight via reinforcement learning. In *NIPS*, volume 16, 2003.
- [40] Jan Peters and Stefan Schaal. Reinforcement learning of motor skills with policy gradients. *Neural networks*, 21(4):682–697, 2008.
- [41] Dean A Pomerleau. Alvin: An autonomous land vehicle in a neural network. In *Advances in Neural Information Processing Systems*, pages 305–313, 1989.
- [42] Sachin Ravi and Hugo Larochelle. Optimization as a model for few-shot learning. In *Under Review, ICLR*, 2017.
- [43] Danilo Jimenez Rezende, Shakir Mohamed, Ivo Danihelka, Karol Gregor, and Daan Wierstra. One-shot generalization in deep generative models. *International Conference on Machine Learning (ICML)*, 2016.
- [44] Stéphane Ross, Geoffrey J Gordon, and Drew Bagnell. A reduction of imitation learning and structured prediction to no-regret online learning. In *AISTATS*, volume 1, page 6, 2011.
- [45] Andrei A Rusu, Neil C Rabinowitz, Guillaume Desjardins, Hubert Soyer, James Kirkpatrick, Koray Kavukcuoglu, Razvan Pascanu, and Raia Hadsell. Progressive neural networks. *arXiv preprint arXiv:1606.04671*, 2016.
- [46] Fereshteh Sadeghi and Sergey Levine. (cad)² rl: Real single-image flight without a single real image. 2016.
- [47] Adam Santoro, Sergey Bartunov, Matthew Botvinick, Daan Wierstra, and Timothy Lillicrap. Meta-learning with memory-augmented neural networks. In *International Conference on Machine Learning (ICML)*, 2016.
- [48] Stefan Schaal. Is imitation learning the route to humanoid robots? *Trends in cognitive sciences*, 3(6):233–242, 1999.
- [49] Jurgen Schmidhuber. Evolutionary principles in self-referential learning. *On learning how to learn: The meta-meta-... hook.) Diploma thesis, Institut f. Informatik, Tech. Univ. Munich*, 1987.
- [50] Jürgen Schmidhuber. Learning to control fast-weight memories: An alternative to dynamic recurrent networks. *Neural Computation*, 1992.
- [51] John Schulman, Sergey Levine, Pieter Abbeel, Michael I Jordan, and Philipp Moritz. Trust region policy optimization. In *ICML*, pages 1889–1897, 2015.
- [52] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [53] Nitish Srivastava, Geoffrey E Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

- [54] Bradlie Stadie, Pieter Abbeel, and Ilya Sutskever. Third person imitation learning. In *Int. Conf. on Learning Representations (ICLR)*, 2017.
- [55] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In *Advances in neural information processing systems*, pages 3104–3112, 2014.
- [56] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*, volume 1. MIT press Cambridge, 1998.
- [57] Gerald Tesauro. Temporal difference learning and td-gammon. *Communications of the ACM*, 38(3):58–68, 1995.
- [58] Sebastian Thrun and Lorian Pratt. *Learning to learn*. Springer Science & Business Media, 1998.
- [59] Eric Tzeng, Judy Hoffman, Ning Zhang, Kate Saenko, and Trevor Darrell. Deep domain confusion: Maximizing for domain invariance. *arXiv preprint arXiv:1412.3474*, 2014.
- [60] Eric Tzeng, Coline Devin, Judy Hoffman, Chelsea Finn, Xingchao Peng, Pieter Abbeel, Sergey Levine, Kate Saenko, and Trevor Darrell. Towards adapting deep visuomotor representations from simulated to real environments. *arXiv preprint arXiv:1511.07111*, 2015.
- [61] Oriol Vinyals, Charles Blundell, Tim Lillicrap, Daan Wierstra, et al. Matching networks for one shot learning. In *Neural Information Processing Systems (NIPS)*, 2016.
- [62] Jane X Wang, Zeb Kurth-Nelson, Dhruva Tirumala, Hubert Soyer, Joel Z Leibo, Remi Munos, Charles Blundell, Dhharshan Kumaran, and Matt Botvinick. Learning to reinforcement learn. *arXiv preprint arXiv:1611.05763*, 2016.
- [63] Kelvin Xu, Jimmy Ba, Ryan Kiros, Kyunghyun Cho, Aaron C Courville, Ruslan Salakhutdinov, Richard S Zemel, and Yoshua Bengio. Show, attend and tell: Neural image caption generation with visual attention. In *ICML*, volume 14, pages 77–81, 2015.
- [64] Jun Yang, Rong Yan, and Alexander G Hauptmann. Cross-domain video concept detection using adaptive svms. In *Proceedings of the 15th ACM international conference on Multimedia*, pages 188–197. ACM, 2007.
- [65] Fisher Yu and Vladlen Koltun. Multi-scale context aggregation by dilated convolutions. In *International Conference on Learning Representations (ICLR)*, 2016.
- [66] B. Ziebart, A. Maas, J. A. Bagnell, and A. K. Dey. Maximum entropy inverse reinforcement learning. In *AAAI Conference on Artificial Intelligence*, 2008.

A 说明性示例：粒子到达

粒子到达问题是一类非常简单的任务。在每个任务中，我们控制一个点机器人到达一个特定的地标，不同任务由不同的地标来区分。如图 1 所示，一个任务可能是到达橙色方块，另一个任务可能是到达绿色三角形。智能体接收自身的二维位置以及每个地标的二维位置。在每个任务中，智能体的初始位置以及所有地标的位置可以在任务的不同实例中变化。

在没有演示的情况下，机器人不知道应该到达哪个地标，因此无法完成任务。因此，这种设置已经触及了一次性模仿的本质，即通过演示来传达任务。学习之后，智能体应该能够从演示中识别出目标地标，并在任务的新实例中到达相同的地标。

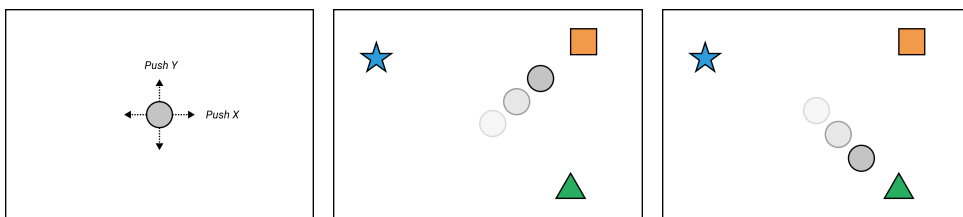


图 1：机器人是一个受二维力控制的质点。任务族是到达一个目标地标。地标的身份因任务而异，模型必须根据演示确定要追求哪个目标。（左）机器人示意图；（中）任务是到达橙色方块，（右）任务是到达绿色三角形。

我们考虑三种架构来解决这个问题：

- **普通 LSTM：**第一种架构是一个具有 512 个隐藏单元的简单 LSTM。它读取演示轨迹，其输出随后与当前状态拼接，并馈送到多层感知器（MLP）以产生动作。
- **带注意力的 LSTM：**在这种架构中，LSTM 从演示序列中输出对不同地标的加权。然后，它在测试场景中应用该加权，并根据当前状态产生地标位置的加权组合。这个二维输出随后与当前智能体位置拼接，并馈送到 MLP 以产生动作。
- **带注意力的最终状态：**该架构并不观察整个演示轨迹，而是仅关注演示中的最终状态（这已足以传达任务），并生成对地标的加权。随后，其处理方式与前述架构相同。

值得注意的是，这三种架构对特定粒子到达场景的专用化程度依次提高，这暗示了表达力与泛化能力之间可能存在权衡。实验结果如图 2 所示。我们观察到，随着架构专用化程度的提高，泛化性能显著提升。对于这一简单任务，以整个演示为条件似乎会损害泛化性能，而仅以最终状态为条件即使没有显式正则化也能取得最佳效果。这符合直觉，因为最终状态已足以刻画当前任务。然而，当任务变得更加复杂时，这一结论似乎不再成立，正如正文中的积木堆叠任务所示。图 3 展示了为粒子到达任务设计的三种架构在不同地标数量下的学习曲线，通过在 100 种不同配置上运行策略，并计算训练和测试数据上的成功率。我们可以清楚地观察到，两种基于 LSTM 的架构随着地标数量的增加表现出过拟合现象。另一方面，使用注意力机制明显提升了泛化性能，并且当仅以最终状态为条件时，在所有场景下都实现了完美的泛化。同样有趣的是，

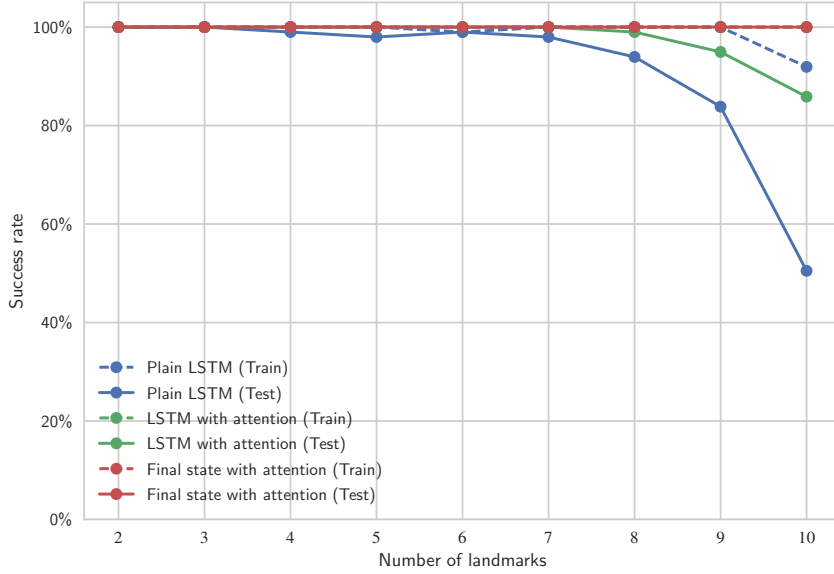


图 2: 粒子到达任务中不同架构的成功率。“训练”曲线表示在训练期间见过的演示条件下, 并在训练期间见过的初始条件下运行策略的成功率; “测试”曲线表示在新轨迹条件下并在新情境中运行策略的成功率。两种基于注意力的架构均达到完美的训练成功率, 曲线重叠。

学习经历了一个相变过程。直观上, 这可能是网络正在学习从演示中推断任务的阶段。一旦这一过程完成, 控制策略的学习几乎变得微不足道。

表 1 和表 2 给出了具体的性能数值以供参考。

#Landmarks	Plain LSTM	LSTM with attention	Final state with attention
2	100.0%	100.0%	100.0%
3	100.0%	100.0%	100.0%
4	100.0%	100.0%	100.0%
5	100.0%	100.0%	100.0%
6	99.0%	100.0%	100.0%
7	100.0%	100.0%	100.0%
8	100.0%	100.0%	100.0%
9	100.0%	100.0%	100.0%
10	99.9%	100.0%	100.0%

表 1: 粒子到达任务在已见演示条件下的成功率, 以及在已见环境中的运行结果。

initial configurations.

B 积木堆叠的补充细节

B.1 架构的完整描述

我们现在用伪代码来具体说明该架构。我们省略了涉及处理小批量演示和观测-动作对, 以及处理不同维度数据所需的填充和掩码的实现细节。我们对所有全连接和卷积操作使用带数据依赖初始化的权重归一化 (Salimans 和 Kingma [2016])。

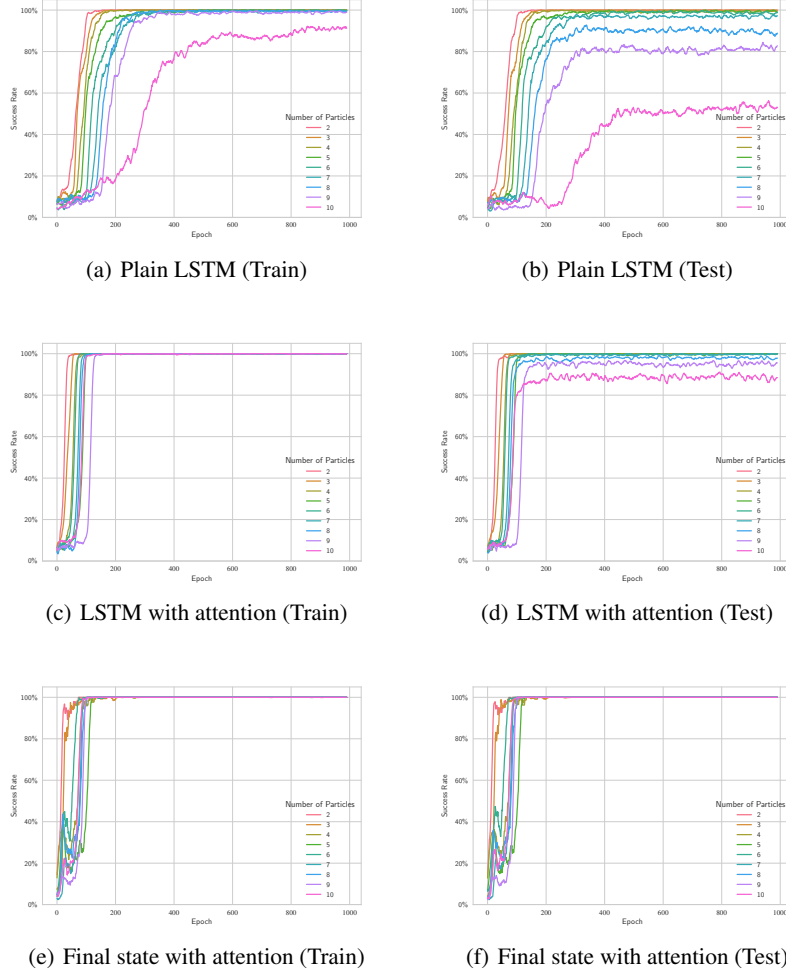


图 3: 粒子到达任务的学习曲线。所示成功率为过去 10 个训练轮次的移动平均值, 以使曲线更平滑。每个策略最多训练 1000 个训练轮次, 使用 Titan X Pascal GPU 最多需要一小时 (从图中可以看出, 大多数实验可以更早完成)。

B.1.1 演示网络

假设演示有 T 个时间步, 并且我们有 B 个积木。我们的架构仅利用输入演示中的观测, 而不利用动作。每个观测是 $a(3B + 2)$ 维向量, 包含每个积木相对于夹爪当前位置的 (x, y, z) 坐标, 以及一个表示夹爪开合状态的 2 维夹爪状态。

完整的操作序列在模块 1 中给出。我们首先应用正文中描述的时间随机失活。然后将观测拆分为关于积木的信息和关于机器人的信息, 其中第一维是时间, 第二维是积木 ID。机器人状态被广播到不同的积木上。因此输出的形状应为 $T \times B \times 3$ 和

$\tilde{T} \times B \times 2$ 分别地。

然后, 我们对块状态执行 1×1 卷积, 将其投影到与每块嵌入相同的维度。接着, 我们执行一系列邻域注意力操作和 1×1 卷积, 其中卷积的输入是注意力结果、

Module 1 Demonstration Network

Input: Demonstration $d \in \mathbb{R}^{T \times (3B+2)}$

Hyperparameters: $p = 0.95, D = 64$

Output: Demonstration embedding $\in \mathbb{R}^{\tilde{T} \times B \times D}$, where $\tilde{T} = \lceil T(1-p) \rceil$ is the length of the down-sampled trajectory.

$d' \leftarrow \text{TemporalDropout}(d, \text{probability}=p)$

块状态, 机器人状态 分割 (d') h 一维卷积 (块状态, 核 size=1,

channels=D)

对 $a \in \{1, 2, 4, 8\}$ **do** ~~Residual connection~~ 修正线性单元 (h) 注意力结果 邻域注意力 (h') h' 拼接 ({ h', 块状

态, 机器人状态 }, axis=-1) h' 一维卷积 (h', 核 size=2, channels=D, dilation=a)

h' 修正线性单元 (h')

$\leftarrow$

$\leftarrow$

$h + h'$

结束循环 演示嵌入 h

当前块位置以及机器人状态。这使得每个块能够查询其他块的状态, 并结合自身状态和机器人状态对查询结果进行推理。在此过程中我们使用了残差连接。

B.1.2 上下文网络

伪代码如模块 2 所示。我们对演示执行一系列注意力操作, 随后对当前状态执行注意力操作, 并通过一个长短期记忆网络 (LSTM) 反复应用这些操作, 每个时间步使用不同的权重 (我们发现这样略微更容易优化)。最后, 我们应用一个最终的注意力操作, 产生一个固定维度的嵌入, 该嵌入与演示长度或环境中方块数量无关。

B.1.3 操作网络

给定上下文嵌入, 该模块只是一个多层感知机。伪代码如模块 3 所示。

B.2 评估置换不变性

在训练和之前的评估中, 我们只从每个等价类中选择一个任务, 其中如果两个任务在置换不同方块后相同, 则认为它们是等价的。这是基于我们的架构对不同方块之间的置换具有不变性的假设。例如, 如果策略仅在任务 abcd 上训练, 那么在给定任务 dcba 的单个演示的情况下, 它应该在任务 dcba 上表现良好。我们现在通过固定一个训练任务, 并评估策略在其所有等价置换下的性能来实验验证这一性质。如图 4 所示, 尽管策略只见过任务 abcd, 但它在所有其他等价任务上达到了相同的性能水平。

B.3 集成的影响

我们现在评估在评估期间采样多个下采样演示的重要性。图 5 显示了在所有训练和测试任务上的性能, 集成数量从 1 到 20 变化。我们观察到, 对于阶段较少的任务, 更多的集成帮助最大。另一方面, 对于较难的任务, 它持续提升性能, 尽管差距较小。我们怀疑这是因为策略已经学会关注演示轨迹中方块已经堆叠在一起的帧。例如, 在只有 1 个阶段的任务中, 这些帧在单个下采样演示中很容易被丢弃。另一方面, 在阶段较多的任务中, 它对缺失帧的鲁棒性更强。使用超过 10 个

Module 2 Context Network

Input: Demonstration embedding $h_{in} \in \mathbb{R}^{\tilde{T} \times B \times D}$, current state $s \in \mathbb{R}^{3B+2}$

Hyperparameters: $D = 64, t_{lstm} = 4, H = 2$

Output: Context embedding $\in \mathbb{R}^{2+6H}$

// Split the current state into block state $\in \mathbb{R}^{B \times 3}$ and robot state broadcasted to all blocks $\in \mathbb{R}^{B \times 2}$

block_state, robot_state $\leftarrow$ SplitSingle(s)

// Initialize LSTM output $\in \mathbb{R}^{B \times D}$ and state (including hidden and cell state) $\in \mathbb{R}^{B \times 2D}$

output, state $\leftarrow$ InitLSTMState(size=B, hidden_dim=D)

for t = 1 to t_{lstm} **do**

 // Temporal attention: every block attend to the same time step

 x $\leftarrow$ output

if t > 1 **then**

 ReLU(x)

结束条件 // 计算用于对演示进行注意力的查询 $\in \mathbb{R}^{B \times D}$ q 全连接层 (x, 输出 dim=D) // 根据注意力计算结果

$\in \mathbb{R}^{H \times B \times D}$ temp SoftAttention(query=q, context=h 输入, memory=h 输入, 数量 heads=H) // 将结果重组为指

定形状 $B \times (HD)$ temp 重塑 (转置 (temp, (1, 0, 2)), (B, H*D)) // 空间注意力: 每个块分别关注不同的块 x 输出

如果 t > 1 则 x 修正线性单元 (x) 结束条件 x 拼接 ({ x, temp }, axis=-1) // 计算用于对当前状态进行注意力的

上下文 $\in \mathbb{R}^{B \times D}$ ctx 全连接层 (x, 输出 dim=D) // 计算用于对当前状态进行注意力的查询 $\in \mathbb{R}^{B \times D}$ q 全连

接层 (x, 输出 dim=D) // 计算用于对当前状态进行注意力的记忆 $\in \mathbb{R}^{B \times (HD+3)}$ mem 拼接 ({ 块状态, temp },

axis=-1) // 根据注意力计算结果 $\in \mathbb{R}^{B \times H \times (HD+3)}$ spatial SoftAttention(query=q, context=ctx, memory=mem, 数

量 heads=H) // 将结果重组为指定形状 $B \times H(HD+3)$ spatial 重塑 (spatial, (B, H*(H*D+3))) // 形成长短期

记忆单元的输入 $\in \mathbb{R}^{D \times (H(HD+3)+H(D+3))}$ input 拼接 ({ 机器人状态, 块状态, spatial, temp }, axis=-1) // 使用

非共享权重运行一步长短期记忆 (即每个时间步使用不同的权重) 输出, 状态 LSTMOneStep(input=input,

state=state) 结束循环 // 对当前状态进行最终注意力, 将 $O(B)$ 表示压缩至 $O(1)$ // 计算查询向量。我们使用

一个固定的、可训练的查询向量, 与输入数据无关, 大小为 $\in \mathbb{R}^{2 \times D}$ (我们使用两个查询, 最初意图是一个

用于源块, 一个用于目标块) q 获取固定查询 0 // 获取注意力结果, 其形状应为 $2 \times H \times 3$ r

SoftAttention(query=q, context=output, memory=block 状态, 数量 heads=H) // 形成最终上下文嵌入 (我们选择

第一个机器人状态, 因为此处无需广播) 上下文嵌入 拼接 ([机器人状态 [0], 重塑 (r, 2*H*3)])

$\leftarrow$

$\leftarrow$

Module 3 Manipulation Network

Input: Context embedding $h_{in} \in \mathbb{R}^{2+6H}$

Hyperparameters: $H = 2$

Output: Predicted action distribution $\in \mathbb{R}^{|A|}$

$h \leftarrow \text{ReLU}(\text{Dense}(h_{in}, \text{output_dim}=256))$

$h \leftarrow \text{ReLU}(\text{Dense}(h, \text{output_dim}=256))$

$\text{action_dist} \leftarrow \text{Dense}(h, \text{output_dim}=|A|)$

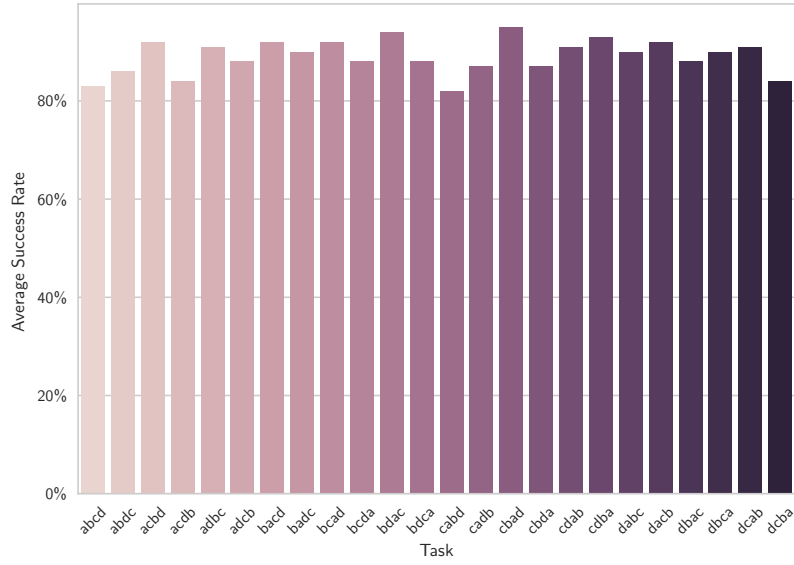


图 4: 策略在一组在排列意义下等价的任务上的性能。

集成似乎没有带来显著改进，因此我们在主要评估中使用了 10 个集成。

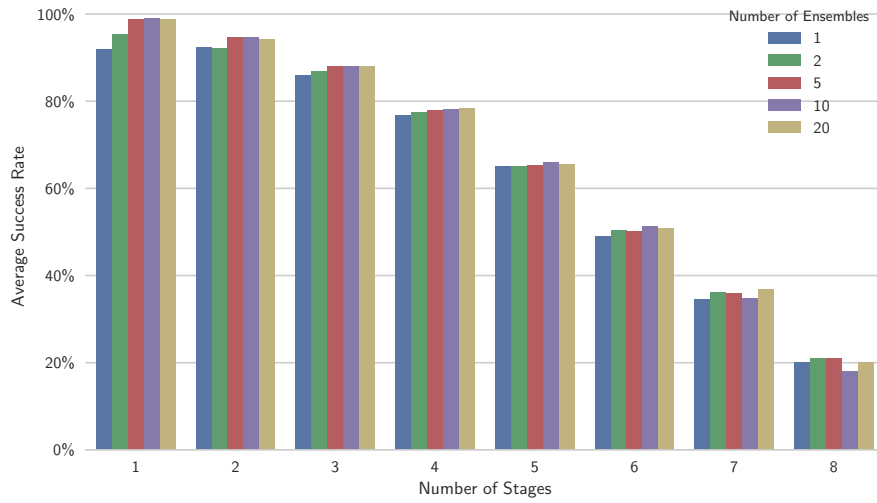


图 5: 不同集成数量的性能。

B.4 失败案例分解

为理解当前方法的局限性，本文对失败案例进行了分解分析。本文考虑了三种失败情形：“错误移动”指策略安排了与期望布局不兼容的布局。这可能是由于策略误解了演示，或由于意外的错误移动将积木打乱成错误布局。“操作失败”指策略造成了不可恢复的失败，例如积木被震落桌面，而当前硬编码策略无法处理这种情况。“可恢复失败”指策略在完成任务前耗尽时间，这可能是由于操作过程中的意外失败，若给予更多时间本可恢复。如图 6 所示，仅以最终状态为条件相比其他架构会产生更多错误移动。除此之外，大多数失败案例实际上源于大多不可恢复的操作失败。¹ 这表明需要习得更好的操作技能，以使学习到的一次性策略更加可靠。

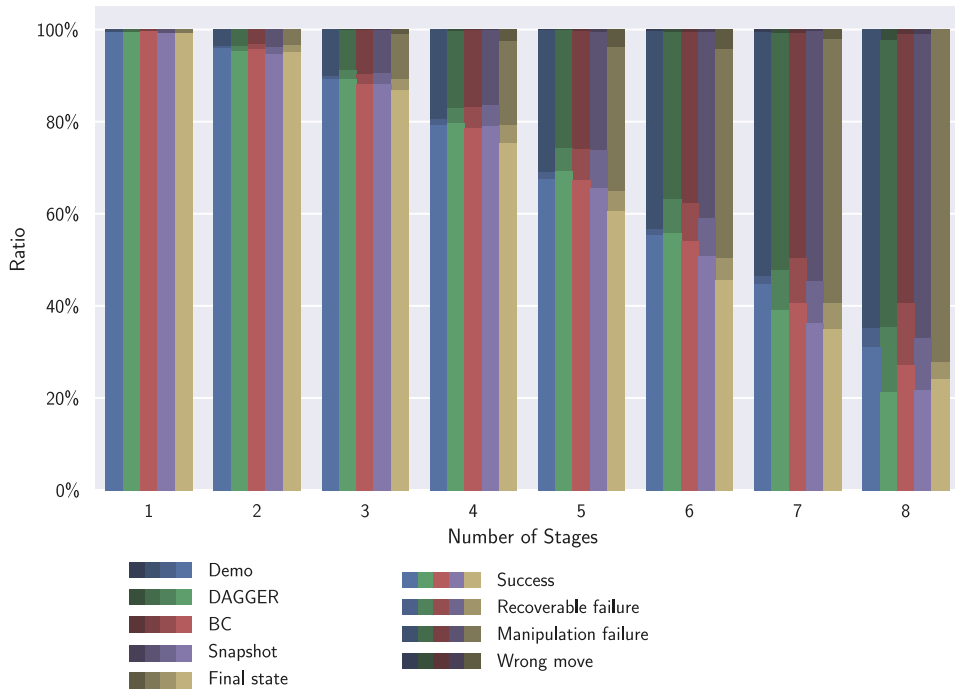


图 6：成功与失败情形的分解。每种颜色所占区域代表相应情形的比例。

B.5 学习曲线

图 7 展示了为积木堆叠任务设计的不同架构的学习曲线。这些学习曲线并不反映最终性能：对于每个评估点，本文从训练数据中采样任务和演示，将环境重置到某一特定阶段的起点（因此部分积木已经堆叠），并仅运行策略至多一个阶段。若训练算法为 DAGGER，这些采样轨迹会被标注并加入训练集。因此该评估不评估泛化能力。本文未在训练过程中进行完整评估，因为这非常耗时：每次评估需要数万次¹ 注意，被误解演示的实际比例可能不同，因为导致操作失败的运行若成功执行，之后可能引发错误移动。另一方面，通过目视检查视频，本文观察到大多数归类为“错误移动”的轨迹实际上源于操作失败（仅以最终状态为条件的策略除外，它似乎确实偶尔会执行真正的错误移动）。

在 > 100 个任务上的轨迹。然而，这些图表仍然有助于反映一些相对趋势。从这些图表中，我们可以观察到，虽然以完整轨迹为条件能获得最佳性能（如正文所示），但它需要更长的训练时间，这仅仅是因为以整个演示为条件需要更多的计算。此外，这也可能是由于对演示进行下采样导致训练过程的高方差，以及网络需要学习正确分割演示的事实。有趣的是，以快照为条件似乎比仅以最终状态为条件学习得更快，这再次表明以中间信息为条件不仅对最终策略有帮助，而且有助于促进训练。我们还观察到，学习在初始阶段最为迅速，而在后期阶段则慢得多，因为操作在后期阶段变得更具挑战性。此外，具有更多阶段的任务较少，因此在评估期间后期阶段的采样频率不如早期阶段高。

B.6 精确性能数值

精确性能数值供参考：

- 表 3 和表 4 分别展示了不同架构在训练任务和测试任务上的成功率；
- 表 5 展示了当集成数量变化时所有任务的成功率；
- 表 6 展示了与 abcd 在排列上等价的任务的成功率；
- 表 7、表 8、表 9、表 10 和表 11 展示了所有考虑架构的不同成功和失败场景的细分。

B.7 更多可视化

图 8 和图 9 展示了注意力权重的完整热力图集。有趣的是，在图 8 中，我们观察到策略并没有像我们最初预期的那样一次关注两个块，而是学会了主要一次只关注一个块。这是合理的，因为在单次堆叠操作的抓取和放置阶段，策略只需要关注夹爪应朝向的那个块。作为背景，图 10 和图 11 展示了神经网络策略执行任务的关键帧。

参考文献

Tim Salimans and Diederik P Kingma. Weight normalization: A simple reparameterization to accelerate training of deep neural networks. In *Advances in Neural Information Processing Systems*, pages 901–901, 2016.

#Landmarks	Plain LSTM	LSTM with attention	Final state with attention
2	100.0%	100.0%	100.0%
3	100.0%	100.0%	100.0%
4	99.0%	100.0%	100.0%
5	98.0%	100.0%	100.0%
6	99.0%	100.0%	100.0%
7	98.0%	100.0%	100.0%
8	93.9%	99.0%	100.0%
9	83.8%	94.9%	100.0%
10	50.9%	85.9%	100.0%

表 2: 以未见演示为条件的粒子到达成功率, 并在

unseen initial configurations.

#Stages	Demo	DAGGER	BC	Snapshot	Final state
1	99.1%	99.1%	99.1%	97.2%	98.8%
2	95.6%	94.3%	93.7%	92.6%	86.7%
3	88.5%	88.0%	86.9%	86.7%	84.8%
4	78.6%	78.2%	76.7%	76.4%	71.9%
5	67.3%	65.9%	65.4%	62.5%	60.6%
6	55.7%	51.5%	52.4%	47.0%	43.6%
7	42.8%	34.3%	37.5%	31.4%	31.5%

表 3: 不同架构在堆叠训练任务上的成功率。

#Stages	Demo	DAGGER	BC	Snapshot	Final state
2	95.8%	94.9%	95.9%	92.8%	94.1%
4	77.6%	77.0%	74.8%	77.2%	75.8%
5	65.9%	65.9%	64.3%	61.1%	51.9%
6	49.4%	50.6%	46.5%	42.6%	35.9%
7	46.5%	36.5%	38.5%	32.8%	32.0%
8	29.0%	18.0%	24.0%	19.0%	20.0%

表 4: 不同架构在积木堆叠测试任务上的成功率。

#Stages	1 Ens.	2 Ens.	5 Ens.	10 Ens.	20 Ens.
1	91.9%	95.4%	98.8%	99.1%	98.7%
2	92.3%	92.2%	94.5%	94.6%	94.1%
3	86.0%	86.8%	87.9%	88.0%	87.9%
4	76.6%	77.4%	77.9%	78.0%	78.3%
5	65.1%	65.0%	65.3%	65.9%	65.5%
6	49.0%	50.4%	50.1%	51.3%	50.8%
7	34.4%	36.1%	36.0%	34.9%	36.8%
8	26.0%	21.0%	21.6%	18.0%	20.0%

表 5: 使用以 DAGGER 策略为条件的集成方法, 不同集成数量下的成功率。

full trajectories, across both training and test tasks.

Task ID	Success Rate
abcd	83.0%
abdc	86.0%
acbd	92.0%
acdb	84.0%
adbc	91.0%
adcb	88.0%
bacd	92.0%
badc	90.0%
bcad	92.0%
bcda	88.0%
bdac	94.0%
bdca	88.0%
cabd	82.0%
cadb	87.0%
cbad	95.0%
cbda	87.0%
cdab	91.0%
cdba	93.0%
dabc	90.0%
dacb	92.0%
dbac	88.0%
dbca	90.0%
dcab	91.0%
dcba	84.0%

表 6: 使用 DAGGER 策略, 一组在置换意义下等价的任务的成功率。

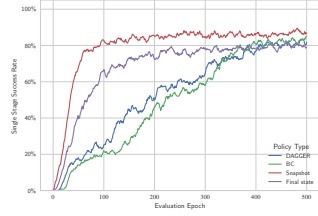
policy conditioned on full trajectories.

#Stages	Success	Recoverable failure	Manipulation failure	Wrong move
1	99.3%	0.0%	0.7%	0.0%
2	95.9%	0.4%	3.7%	0.0%
3	89.1%	0.7%	10.1%	0.1%
4	79.2%	1.2%	19.4%	0.1%
5	67.5%	1.4%	30.9%	0.2%
6	55.2%	1.4%	43.1%	0.3%
7	44.6%	1.7%	53.2%	0.6%
8	36.9%	1.3%	64.9%	0.0%

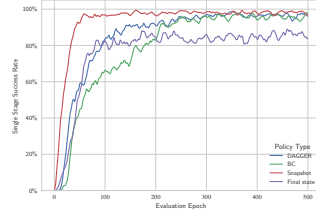
表 7: DAGGER 策略的成功与失败场景分解。

#Stages	Success	Recoverable failure	Manipulation failure	Wrong move
1	99.4%	0.0%	0.6%	0.0%
2	95.3%	0.9%	3.8%	0.0%
3	89.1%	1.9%	8.8%	0.1%
4	79.5%	3.5%	16.7%	0.3%
5	69.1%	5.0%	25.6%	0.3%
6	55.8%	7.3%	36.4%	0.5%
7	39.0%	8.6%	51.5%	0.8%
8	21.2%	14.1%	62.4%	2.4%

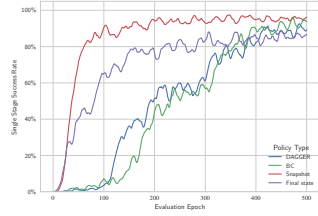
表 8: DAGGER 策略的成功与失败场景分解。



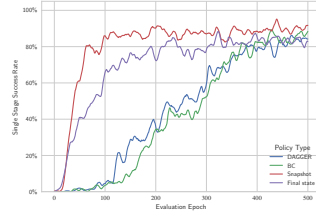
(a) All Stages



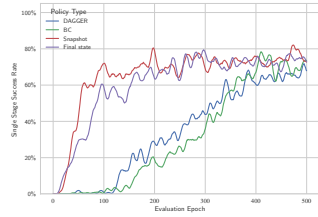
(b) Stage 0



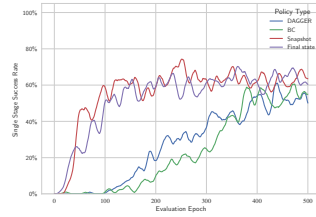
(c) Stage 1



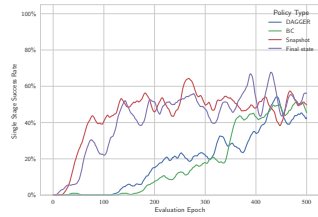
(d) Stage 2



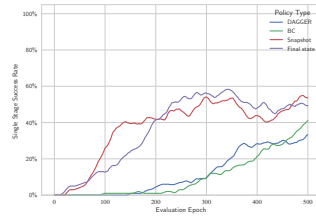
(e) Stage 3



(f) Stage 4



(g) Stage 5



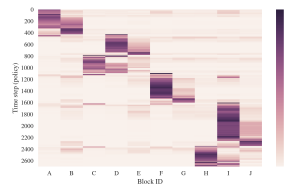
(h) Stage 6

图 7: 积木堆叠任务的学习曲线。第一幅图展示了所有阶段初始配置上的平均成功率。后续各图展示了每个阶段的分解情况。例如，“阶段 3”表示前 3 个堆叠操作已完成，策略在完成第 4 个堆叠操作的能力上进行评估。

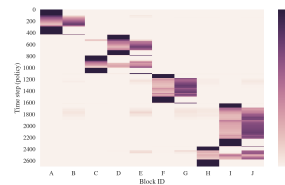
#Stages	Success	Recoverable failure	Manipulation failure	Wrong move
1	99.6%	0.0%	0.4%	0.0%
2	95.6%	1.1%	3.2%	0.1%
3	88.1%	2.2%	9.5%	0.2%
4	78.5%	4.5%	16.8%	0.2%
5	67.2%	6.6%	25.7%	0.4%
6	53.9%	8.3%	37.1%	0.6%
7	40.6%	9.8%	48.7%	0.9%
8	27.0%	18.3%	58.4%	1.1%

#Stages	Success	Recoverable failure	Manipulation failure	Wrong move
1	99.1%	0.0%	0.9%	0.0%
2	94.5%	1.6%	3.8%	0.1%
3	88.0%	2.5%	9.3%	0.2%
4	78.9%	4.6%	16.2%	0.3%
5	65.6%	8.0%	25.8%	0.6%
6	50.8%	8.3%	40.2%	0.7%
7	36.1%	9.2%	54.2%	0.4%
8	21.6%	11.4%	65.9%	1.1%

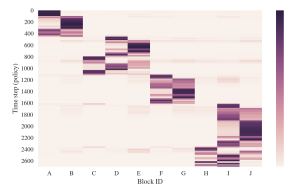
#Stages	Success	Recoverable failure	Manipulation failure	Wrong move
1	99.2%	0.0%	0.8%	0.0%
2	95.1%	1.3%	3.6%	0.0%
3	86.7%	2.5%	9.7%	1.1%
4	75.2%	4.0%	18.3%	2.5%
5	60.5%	4.3%	31.2%	4.0%
6	45.5%	4.7%	45.5%	4.3%
7	34.9%	5.6%	57.3%	2.2%
8	24.1%	8.6%	72.3%	0.0%



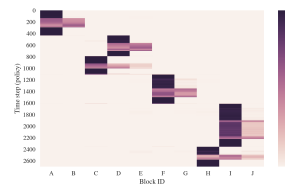
(a) Head 0



(b) Head 1



(c) Head 2



(d) Head 3

图 8: 所有 4 个查询头在不同块上的注意力权重热力图。

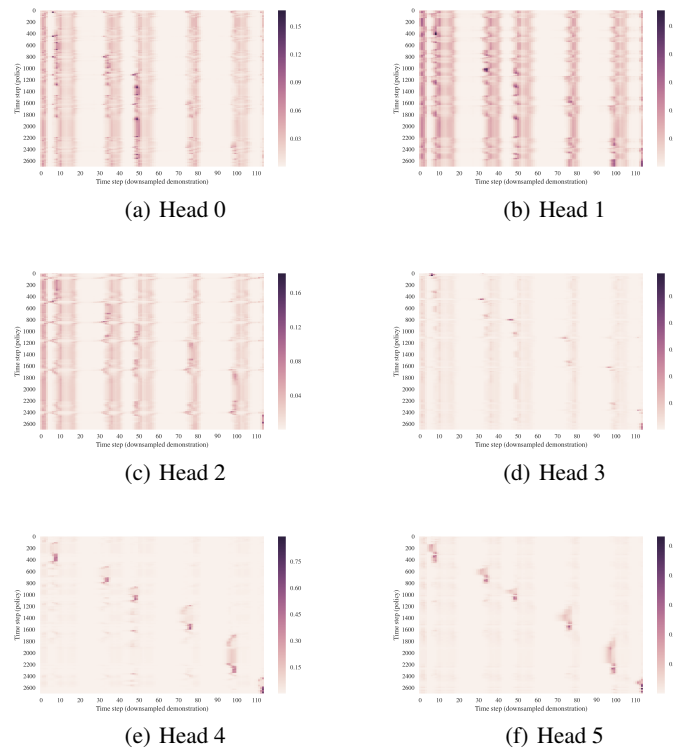


图 9：所有 6 个查询头在下采样演示轨迹上的注意力权重热力图。每个长短期记忆步骤有 2 个查询头，共执行 3 个长短期记忆步骤。

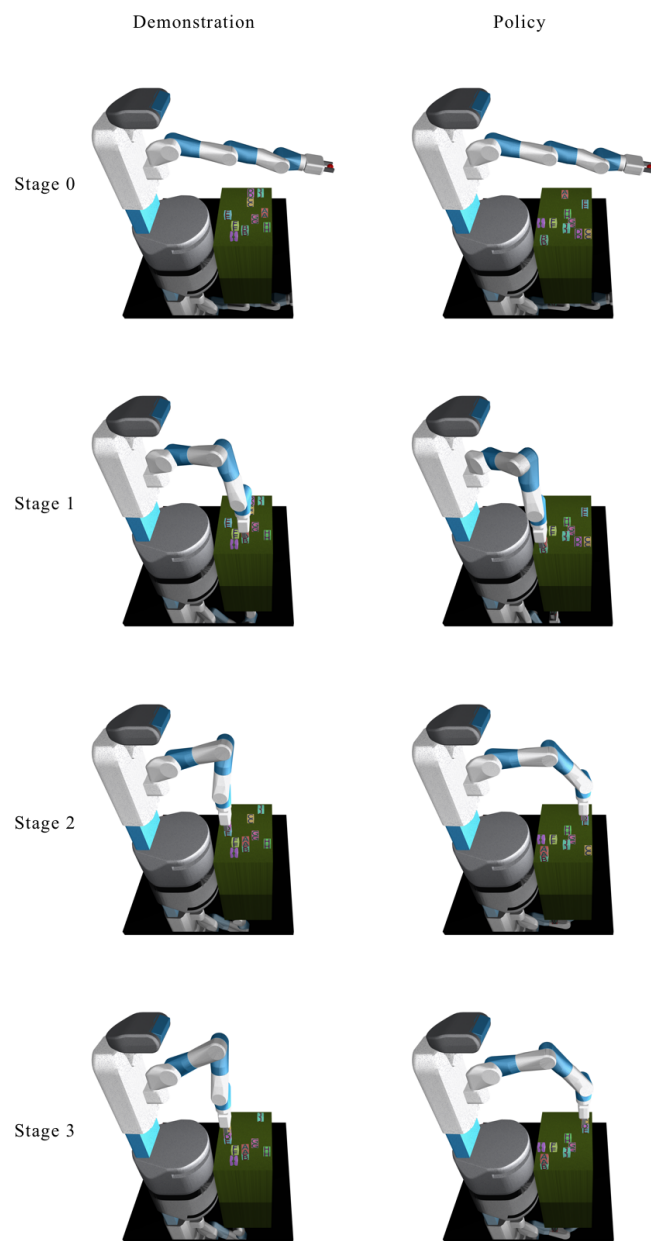


图 10: 用于注意力热力图可视化的任务示意图（前半部分）。任务为 ab cde fg hij。左侧展示演示中的关键帧。右侧展示策略在观看完整演示后，如何在相同任务的新初始化中重现相同布局。

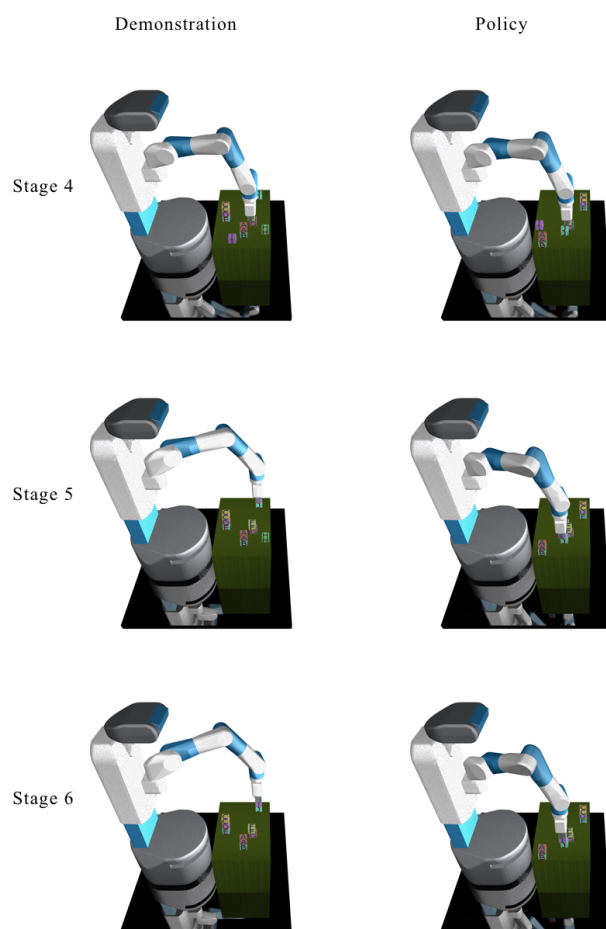


图 11：用于注意力热力图可视化的任务示意图（后半部分）。任务为 ab cde fg hij。左侧展示演示中的关键帧。右侧展示策略在观看完整演示后，如何在相同任务的新初始化中重现相同布局。